

Versión 3 — selección de arquitectura y resultados finales

Índice general

Versión 3 — selección de arquitectura, datos dirigidos y resultados finales	1
Resultado	1
1. Etapa 1 — selección de arquitectura	2
2. Etapa 2 — datos y régimen de entrenamiento	3
3. Etapa 3 — hiperparámetros, modelo final y evaluación	4
4. Deficiencias de la auditoría, corregidas	7
5. Prototipo	7
6. Qué cambió de verdad, y qué no	8
Anexo. Reproducibilidad	8

Versión 3 — selección de arquitectura, datos dirigidos y resultados finales

Autor: Willian Alexander Chávez Servellón **Fecha:** 12 de septiembre de 2026 **Objetivo fijado:** F1-macro ≥ 0.80 **con el extremo inferior del intervalo de confianza también ≥ 0.80** , corrigiendo las deficiencias de la auditoría 2v. **Taxonomía:** las mismas cuatro clases de siempre. No se añadió ninguna.

Resultado

	Valor
F1-macro sobre test.csv (n = 460)	0.8499
IC 95 % bootstrap	[0.8164, 0.8824]
Accuracy	0.8522 (392/460)
F1-macro validación	0.8409
MCC · QWK · ECE	0.7996 · 0.8087 · 0.1214

La meta se cumple en el criterio duro. El extremo inferior del intervalo está en 0.8164, dieciséis milésimas por encima del umbral. No es el punto estimado el que pasa: pasa el intervalo entero.

Contra los modelos de la entrega anterior, medidos con el mismo código sobre el mismo test:

Modelo	F1-macro	IC 95 %	Aciertos	¿Meta dura?
V3 final (RoBERTuito)	0.8499	[0.8164, 0.8824]	392/460	sí
TwitterXLM-R sobre B2	0.8302	[0.7940, 0.8633]	383/460	no

Modelo	F1-macro	IC 95 %	Aciertos	¿Meta dura?
XLM-R v1 (Julio, agosto)	0.8058	[0.7665, 0.8420]	371/460	no
XLM-R mitigado (Julio, 2v)	0.7887	[0.7486, 0.8257]	363/460	no
mBERT v1 (Cristian)	0.7777	[0.7363, 0.8144]	360/460	no
mBERT B2 (Cristian, 2v)	0.7610	[0.7172, 0.7983]	353/460	no

McNemar pareado sobre las mismas 460 instancias:

V3 vs XLM-R v1 b = 47, c = 26, p = 0.0186 -> V3
V3 vs XLM-R mitig. b = 52, c = 23, p = 0.0011 -> V3
V3 vs mBERT B2 b = 58, c = 19, p = 0.0000 -> V3

Las tres son significativas. Es la primera vez en el proyecto que una mejora supera la prueba pareada.

1. Etapa 1 — selección de arquitectura

1.1 Adecuación del tokenizador, medida sobre los 2,144 textos reales

Antes de entrenar nada conviene saber cuánto le cuesta a cada modelo *leer* este dialecto. La fertilidad es el número de subtokens por palabra: cuanto más alta, más troceado queda el texto y menos contexto cabe en la ventana.

Modelo	Vocab	Fertilidad	UNK	Jerga como 1 token	Frag. ofuscación
RoBERTuito	30 K	1.470	0	108/1923 (5.6 %)	2.61
BERTIN	50 K	1.546	0	19 (1.0 %)	2.70
BETO	31 K	1.598	588	86 (4.5 %)	2.83
XLM-R base · large · TwitterXLM-R	250 K	1.696	3	45 (2.3 %)	2.48
mBERT	120 K	1.805	45	31 (1.6 %)	2.65
mDeBERTa-v3	250 K	1.851	45	40 (2.1 %)	2.57

Tres lecturas:

- **mBERT era la peor elección posible del grupo.** Gasta un 23 % más de subtokens que RoBERTuito sobre el mismo texto: cerote → cero ##te, maje → maj ##e, pisto → pis ##to. Parte de su desventaja frente a XLM-R estaba en el tokenizador, no en el transformer.
- **BETO queda descartado** por 588 [UNK] sobre el corpus.
- **Pero ningún tokenizador conoce el dialecto.** El mejor cubre el 5.6 % del léxico como unidad. La palanca no era cambiar de tokenizador.

1.2 Bake-off: siete arquitecturas, hiperparámetros idénticos

Modelo	F1-macro val	Parámetros	Tiempo
TwitterXLM-R	0.8289	278 M	3.1 min
RoBERTuito	0.8272	109 M	1.5 min
BERTIN	0.8141	125 M	1.7 min
XLM-R large	0.8090	560 M	42.2 min
BETO	0.8000	110 M	1.6 min
XLM-R base	0.7895	278 M	2.0 min
mBERT	0.7636	178 M	2.1 min

Escalar no sirve. XLM-R large tiene cinco veces los parámetros de RoBERTuito, tarda veintiocho veces más y saca menos F1. Con 2,144 ejemplos reales el cuello de botella no es capacidad del modelo.

El pre-entrenamiento de dominio sí sirve, y mucho. TwitterXLM-R es *la misma arquitectura* que XLM-R base —mismo tamaño, mismo tokenizador, mismos pesos iniciales de referencia— y le saca **+0.039** solo por haber sido preentrenado sobre tuits. Ese es el hallazgo limpio del capítulo: la mejora viene del corpus de preentrenamiento, no del diseño arquitectónico.

2. Etapa 2 — datos y régimen de entrenamiento

2.1 El generador de sintéticos V3

Se construyó sobre el léxico salvadoreño del proyecto, reconstruido en **110 familias canónicas** con sus **1,979 variantes de ofuscación**, que son formas observadas en el corpus y no deformaciones inventadas.

Tres defectos del generador de B2 que se corrigieron al reescribirlo:

- **Concordancia.** Los portadores son vocativos y aposiciones, que no exigen concordancia de género ni número. La primera versión producía «Los mujeres son unos pandilleros»; el modelo habría aprendido ese ruido como señal.
- **Coherencia emoji[flecha izquierda y derecha]grupo.** Cada emoji solo se empareja con referentes de su propia fila del catálogo. Cruzar [jabón] (antisemita) con «chinos» enseñaría una asociación falsa.
- **Tipado de la jerga.** «pisto» es un objeto, «chivo» un adjetivo y «puchica» una interjección. Meterlas en el mismo hueco producía «ya viene el chamba». Cada término lleva su rol y cada rol sus portadores.

Resultado: **2,916 sintéticos nuevos** en diez familias, todas dirigidas a una debilidad medida en la auditoría.

2.2 Composición del corpus V3

```
train 6,202 filas 2,144 reales + 1,142 sintéticos B2 + 2,916 sintéticos V3
val   459 byte a byte idénticos a B2 sha 45aabe9744711d74
test  460 byte a byte idénticos a B2 sha 5d1284efcc7df20f
```

val y test no se tocaron. Ni una etiqueta, ni una normalización. Es lo que mantiene comparables todas las métricas del proyecto desde la v1 y lo que hace imposible la objeción que le hicimos al Prototipo V2.

2.3 La ablación, y un resultado que no esperaba

Variante	TwitterXLM-R	RoBERTuito
B2 solo	0.8331	0.8272
V3, peso 1.0	0.8080	0.8337
V3, peso 0.5	0.8094	0.8371
V3, peso 0.25	0.8144	0.8403
V3, dos fases	0.8090	0.8386
V3, peso 0.5 + dos fases	—	0.8473
B2 + vocabulario extendido	0.7756	0.8231
V3 + vocabulario, peso 0.5	0.8016	0.8213

Dos hallazgos, uno de ellos contra mi propia hipótesis de partida:

Los mismos datos ayudan a un modelo y perjudican al otro. Con TwitterXLM-R (278 M) los sintéticos cuestan hasta 2.5 puntos; con RoBERTuito (109 M) ganan hasta 2. La lectura es que el modelo grande tiene capacidad de sobra para memorizar la estructura de las plantillas, y esa estructura no existe en el lenguaje real; el pequeño no puede memorizarla y se queda con la señal. No es un resultado sobre «los sintéticos», es un resultado sobre la interacción entre capacidad del modelo y regularidad de los datos.

La extensión de vocabulario perjudica siempre. Añadir los 189 tokens de dominio —placeholders de emoji y jerga canónica— cuesta entre 0.6 y 5.8 puntos. La hipótesis era buena: :pistola: se parte en cuatro fragmentos y eso explicaba el 0/12 en emojis de amenaza. Pero un token nuevo nace con un embedding aleatorio y hacen falta muchos más ejemplos de los que hay para aprenderlo; mientras tanto sustituye una representación imperfecta pero informativa por ruido. **Se descarta**, y queda documentado como vía explorada y cerrada.

2.4 Los sintéticos que sobran

Un subconjunto dirigido —solo las familias que atacan una debilidad medida, descartando 1,292 filas de insultos ofuscados y odio ofuscado— da **0.8472** frente a **0.8473** del conjunto completo. Empate.

Es decir: **la mitad de los sintéticos no aportaba nada**. Los modelos ya acertaban entre el 68 % y el 89 % en insultos ofuscados; añadir volumen donde no hay problema solo desplaza la distribución de entrenamiento. Es una lección aplicable al B2 también, y explica por qué aquel costó F1.

3. Etapa 3 — hiperparámetros, modelo final y evaluación

3.1 Búsqueda de hiperparámetros

Veinte pruebas con Optuna (TPE, semilla 42) sobre RoBERTuito, optimizando F1-macro en **validación**. El test no participó ni se miró.

Parámetro	Rango	Elegido
learning rate	8e-6 – 6e-5 (log)	5.07e-5
batch size	8 / 16 / 32	32
épocas	3 – 6	4
weight decay	0.0 – 0.15	0.134
warmup	0.0 – 0.2	0.136
peso de los sintéticos	0.25 / 0.5 / 0.75 / 1.0	1.0
épocas finales solo con reales	0 – 3	3
pesos de clase	sí / no	sí

Mejor prueba: **0.8523** en validación, frente a 0.8272 de la configuración por defecto. La búsqueda aportó **+0.025**, y es el criterio de la rúbrica que llevaba todo el proyecto en cero.

El óptimo encontrado es interesante: peso completo a los sintéticos **más** tres épocas finales solo con ejemplos reales. El modelo aprovecha toda la señal de robustez y después se reasienta en la distribución natural. Es una versión limpia de lo que Cristian intentó a mano con su «masa efectiva ≈ 90 ».

3.2 Modelo final

Entrenado con tres semillas (42, 1337, 2026) y **elegido por la mediana, no por la mejor**: quedarse con la semilla más favorable es sobreajustar al conjunto de selección en silencio.

Semilla	F1-macro val
42	0.8374
1337	0.8409 ← mediana, elegida
2026	0.8555

La dispersión entre semillas es de 1.8 puntos, que es del mismo orden que varias de las «mejoras» que este proyecto ha reportado en entregas anteriores. Vale la pena decirlo en la tesina.

models/v3/final/

```

modelo base      pysentimiento/robertuito-base-cased (109 M parámetros)
sha256 pesos     e79618319c7de5c1497f58de99856f701f3daafee...
contrato         max_length 128 · texto_modelo · normalizador v1.2 · semilla 1337

```

3.3 Métricas por clase

Clase	Precisión	Recall	F1	n
No Tóxico	0.8793	0.8293	0.8536	123
Lenguaje Ofensivo	0.8774	0.8553	0.8662	159
Discurso de Odio	0.8100	0.8526	0.8308	95
Amenazas/Violencia	0.8202	0.8795	0.8488	83

Las cuatro clases pasan de 0.83. Discurso de Odio, que era la peor de todos los modelos anteriores (0.7330 – 0.8041), sube a 0.8308. Amenazas/Violencia pasa de 0.7349 – 0.7975 a 0.8488.

3.4 Cortes

Corte	n	V3	XLM-R v1	mBERT B2
Plataforma X	228	0.8976	0.8673	0.8190
Plataforma Facebook	232	0.7773	0.7154	0.6648
FPR No Tóxico con jerga	29	0.2759	0.4483	0.4138
FPR No Tóxico sin jerga	94	0.1383	0.1596	0.1596
Brecha dialectal	—	+0.1376	+0.2887	+0.2542

El sesgo dialectal baja en datos reales por primera vez en el proyecto. El FPR con jerga pasa de 41–45 % a 27.6 %, y la brecha se reduce a la mitad. La auditoría de la entrega 2v concluía que ni B2 ni el pipeline de Julio movían esta cifra; ahora sí se mueve.

Con la advertencia de siempre: **n = 29**. Un caso vale 3.4 puntos. La tendencia es consistente con el resto de las mediciones, pero ese corte por sí solo no tiene potencia estadística.

La brecha X/Facebook persiste (12 puntos). Es la limitación estructural del corpus que ya se había identificado y que no se resuelve con modelado.

3.5 Batería adversarial (144 casos)

Grupo	n	V3	mBERT B2	XLM-R v1	XLM-R mitig.
A. Ofuscación de groserías	20	20	12	17	11
B. Emojis	18	17	16	6	10
C. Amenazas ofuscadas	11	11	11	5	11
D. Jerga inofensiva	14	13	13	8	6

Grupo	n	V3	mBERT B2	XLm-R v1	XLm-R mitig.
E. Control sin jerga	12	12	12	12	11
F. Falsos amigos dialectales	34	29	30	14	17
G. Pares mínimos de emoji	35	23	26	12	11
Global	144	125	120	74	77

Grupo A pasa a 20/20: ninguna deformación de grosería se le escapa. Grupo G (pares mínimos de emoji) es el punto débil que queda, con 23/35.

Higiene verificada: 0 de los 144 casos aparece en el train. Las siete filas que el proyecto arrastraba desde agosto están fuera.

3.6 Sonda dirigida (204 casos)

Familia	n	V3	mBERT B2	XLm-R v1	mBERT v1
1. Ofuscación de groserías	66	66	45	59	52
2. Amenazas de muerte	72	72	61	26	23
3. Emoji de amenaza	12	12	12	0	0
4. Emoji de odio	4	4	4	4	2
5. Emoji inofensivo	16	16	16	5	3
6. Par mínimo de emoji	14	12	14	2	4
7. Jerga inofensiva	10	9	8	2	2
8. Control estándar	10	9	7	7	6
Global	204	200	167	105	92
Brecha dialectal (7 – 8)	—	0.0000	−0.1000	+0.5000	+0.4000

Advertencia metodológica, y es importante que conste. Cuatro textos de la sonda aparecen en el train del V3 (SND-0072, SND-0137, SND-0193, SND-0203); excluyéndolos el resultado es **196/200**. Pero el problema de fondo no son esos cuatro: el 31 % de la sonda usa términos del mismo léxico salvadoreño con el que se generaron los sintéticos V3. **La sonda dejó de ser un test independiente para este modelo.** Sigue siéndolo para los cuatro modelos anteriores, que no vieron esos datos, y por eso la comparación de la columna sirve; pero el 200/204 del V3 no se puede presentar como generalización pura. La cifra que sí es independiente y limpia es el 0.8499 sobre test.csv.

3.7 Evadibilidad

La auditoría 2v mostró que el filtro de expresiones regulares del Prototipo V2 se desactivaba anteponiendo «gracias». Misma prueba, 100 amenazas, sobre el V3:

Prefijo	V3 → No Tóxico	Prototipo V2 → No Tóxico
(ninguno)	3/100	1/100
gracias,	5/100	20/100
qué onda,	5/100	19/100
todo bien,	7/100	17/100
mi hermano,	5/100	21/100
buena onda,	7/100	17/100

El marcador afectivo degrada al V3 en 2–4 casos de 100, que es la sensibilidad normal de cualquier clasificador al contexto. En el Prototipo V2 lo degradaba en 16–20, porque la regla lo forzaba. **Sin capa de reglas no hay puerta trasera.**

4. Deficiencias de la auditoría, corregidas

#	Deficiencia detectada en la entrega 2v	Estado
1	7 filas compartidas entre la batería adversarial y el train	Corregido. 0 solape verificado
2	m@tar → m@usuario: la anonimización destruía la ofuscación con arroba	Corregido. Guarda de límite de palabra. test.csv no cambia ni una fila
3	[pistola de agua] traducido como :pistola_de_agua:	Corregido a :pistola:
4	Sin optimización de hiperparámetros (20 pts de rúbrica en cero)	Corregido. 20 pruebas Optuna documentadas, +0.025
5	Sin prototipo desplegado (15 pts en cero)	Corregido. FastAPI + interfaz, verificados
6	Sesgo dialectal sin mejora en datos reales	Corregido. FPR con jerga 0.4483 → 0.2759
7	Meta de 0.80 no alcanzada con significancia	Corregido. IC inferior 0.8164
8	Métricas no reproducibles desde los pesos entregados	Corregido. Todo sale de models/v3/final con manifiesto SHA-256

Pendientes que no se resuelven con modelado, y hay que decirlo:

- **La brecha X/Facebook** (0.8976 vs 0.7773) sigue ahí. Es composición del corpus, no arquitectura.
- **La sonda dialectal real tiene n = 29.** Sin recolectar más No Tóxico con jerga, cualquier cifra de sesgo sobre datos reales es frágil.
- **Los emojis de amenaza siguen sin un solo caso real recolectado.** Toda su cobertura es sintética.
- **Calibración imperfecta** (ECE 0.1214). Mejora frente a mBERT B2 (0.1640) pero el prototipo devuelve confianzas del 100 % que no reflejan incertidumbre real.

5. Prototipo

app/api/main.py (FastAPI) y app/ui/index.html. Verificado en ejecución:

```
GET /salud          estado, dispositivo, clases y contrato de inferencia
POST /clasificar    un texto -> clase, confianza, las 4 probabilidades, nivel de riesgo
POST /clasificar-lote hasta 64 textos
GET /               interfaz web
```

El endpoint importa normalizar del **mismo módulo** que preparó el entrenamiento. No hay una segunda implementación que pueda divergir: es la condición para que las métricas del informe describan lo que pasa en producción.

Comprobación en vivo:

Entrada	Salida
Qué onda maje, todo bien por allá	No Tóxico
Callate ya, cerote	Lenguaje Ofensivo
Los nicaragüenses son una plaga	Discurso de Odio
Te voy a m4t4r	Amenazas/Violencia
Ya sabes lo que te espera [pistola de agua]	Amenazas/Violencia
gracias, te voy a matar	Amenazas/Violencia

La última fila es el caso que evadía el Prototipo V2.

6. Qué cambió de verdad, y qué no

El proyecto llevaba tres entregas atascado alrededor de 0.78-0.81 y la mejora vino de tres decisiones, en este orden de importancia:

1. **Cambiar de familia de modelo** (+0.04). RoBERTuito y TwitterXLM-R están preentrenados sobre lenguaje de redes sociales en español. mBERT y XLM-R base no. Todo el esfuerzo anterior se hizo con dos modelos mal elegidos para la tarea.
2. **Optimizar hiperparámetros** (+0.025). Nunca se había hecho.
3. **Datos sintéticos dirigidos, usados con el régimen correcto** (+0.01 a +0.02 sobre RoBERTuito). Con el régimen equivocado —peso completo, sin fase final sobre reales— restan.

Y tres cosas que **no** funcionaron, documentadas porque ahorran tiempo a quien venga después:

- **Escalar el modelo.** XLM-R large, 560 M de parámetros, peor que uno de 109 M.
- **Extender el vocabulario.** Perjudica en las seis configuraciones probadas.
- **Generar sintéticos donde no hay problema.** La mitad de los que produce eran prescindibles.

Lo que sigue sin resolverse es lo que la auditoría ya señalaba: **2,144 ejemplos reales**, 29 casos de jerga en el test, cero emojis de amenaza recolectados. El 0.8499 se sostiene, pero el siguiente salto no sale de más arquitectura.

Anexo. Reproducibilidad

```
python3 src/audit/analisis_tokenizadores.py # 1.1
python3 src/v3/bakeoff.py # 1.2
python3 src/v3/construir_dataset.py # 2.2
python3 src/v3/ablacion.py <modelo_hf> <alias> # 2.3
python3 src/v3/hpo.py <modelo_hf> <alias> 20 # 3.1
python3 src/v3/entrenar_final.py # 3.2
python3 src/v3/evaluar.py models/v3/final v3_final # 3.3-3.6
python3 -m uvicorn app.api.main:app # 5
```

Semilla 42 en todo salvo el modelo final (1337, por mediana de semillas).

Artefacto	Ruta
Modelo final + manifiesto SHA-256	models/v3/final/
Corpus V3	data/dataset_v3/
Sintéticos con su plantilla de origen	data/dataset_v3/sinteticos_v3.csv

Artefacto	Ruta
Bake-off, ablaciones, HPO	reports/v3_arquitectura/
Logits, predicciones y métricas finales	reports/v3_resultados/
Auditoría de la entrega 2v	reports/9-12- 2026/Informe_Auditoria_Entrega_2v.md